



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра математического обеспечения и стандартизации информационных
технологий

Отчет по практической работе №2

по дисциплине «Тестирование и верификация программного обеспечения»

Выполнили:

Студенты группы ИКБО-32-23

Команда «ВМР»

Мингазутдинов А.Р.

Ильичев Г.П.

Проверил:

Отчет представлен

«__»_____2025г.

Москва 2025

Содержание

Цель работы.....	3
Задачи	3
Описание функциональности.....	3
Исходный код модуля (с преднамеренной ошибкой)	4
Модульные тесты (pytest).....	6
Исправление ошибки	8
Мутационное тестирование (инструкции).....	11
Результаты.....	11
Вывод	13
Список используемой литературы.....	14

Цель работы

Познакомиться с методами модульного тестирования и мутационного тестирования на примере небольшого программного модуля — генератора паролей с возможностью проверки сложности и подсчёта энтропии.

Задачи

1. Разработать модуль на Python с минимум 5 функциями, одна из которых содержит преднамеренную ошибку (для роли тестировщика).
2. Написать модульные тесты (pytest) для проверки функциональности модуля и выявления ошибки.
3. Исправить ошибку, повторно запустить тесты.
4. Провести (описательно) мутационное тестирование с использованием MutPy (инструкции и пример команд).

Описание функциональности

Модуль реализует:

- Конвертацию температуры между Цельсиями, Фаренгейтами и Кельвинами, с поддержкой правильного округления и обработки граничных значений•
- Перевод длины между метрами, километрами, сантиметрами, миллиметрами и милями
- Перевод массы между граммами, килограммами, тоннами и фунтами
- Проверку корректности входных данных, включая тип значения и допустимость единиц измерения
- Точное округление результатов до заданного числа знаков после запятой для удобного использования в расчетах и отчетах

Исходный код модуля (с преднамеренной ошибкой)

```
def validate_number(value):
    if not isinstance(value, (int, float)):
        raise TypeError(f"Ошибка: ожидалось число, получено {type(value).__name__}")

#ТЕМПЕРАТУРА
def convert_temperature(value, from_unit, to_unit):
    validate_number(value)
    from_unit, to_unit = from_unit.upper(), to_unit.upper()
    if from_unit == "C":
        celsius = value
    elif from_unit == "F":
        celsius = (value - 32) * 5 / 9
    elif from_unit == "K":
        celsius = value - 273.15
    else:
        raise ValueError("Недопустимая единица температуры: " + from_unit)
    if to_unit == "C":
        return round(celsius, 2)
    elif to_unit == "F":
        return round(celsius * 9 / 5 + 32, 2)
    elif to_unit == "K":
        return round(celsius + 273, 2)
    else:
        raise ValueError("Недопустимая единица температуры: " + to_unit)

#ДЛИНА
def convert_length(value, from_unit, to_unit):
    validate_number(value)
    from_unit, to_unit = from_unit.lower(), to_unit.lower()
    to_meters = {
        "m": 1,
        "km": 1000,
        "cm": 0.01,
        "mm": 0.001,
        "mi": 1609.34
    }
    if from_unit not in to_meters or to_unit not in to_meters:
        raise ValueError("Недопустимая единица длины")
    meters = value * to_meters[from_unit]
    result = meters / to_meters[to_unit]
    return round(result, 4)

#МАССА
def convert_mass(value, from_unit, to_unit):
    validate_number(value)
    from_unit, to_unit = from_unit.lower(), to_unit.lower()
    to_grams = {
```

```

    "g": 1,
    "kg": 1000,
    "t": 1_000_000,
    "lb": 453.592
}
if from_unit not in to_grams or to_unit not in to_grams:
    raise ValueError("Недопустимая единица массы")
grams = value * to_grams[from_unit]
result = grams / to_grams[to_unit]
return round(result, 4)
#ОБЩИЙ
def convert(value, from_unit, to_unit):
    from_unit, to_unit = from_unit.lower(), to_unit.lower()
    temperature_units = {"c", "f", "k"}
    length_units = {"m", "km", "cm", "mm", "mi"}
    mass_units = {"g", "kg", "t", "lb"}
    if from_unit in temperature_units and to_unit in temperature_units:
        return convert_temperature(value, from_unit, to_unit)
    elif from_unit in length_units and to_unit in length_units:
        return convert_length(value, from_unit, to_unit)
    elif from_unit in mass_units and to_unit in mass_units:
        return convert_mass(value, from_unit, to_unit)
    else:
        raise ValueError("Невозможно определить тип измерения для заданных единиц.")

```

Модульные тесты (pytest)

```
import pytest
from converter import (
    convert_temperature,
    convert_length,
    convert_mass,
    convert,
)
#ТЕСТЫ ТЕМПЕРАТУРЫ
def test_convert_temperature_c_to_f():
    result = convert_temperature(0, "C", "F")
    assert result == 32.0
def test_convert_temperature_f_to_c():
    result = convert_temperature(212, "F", "C")
    assert result == 100.0
def test_convert_temperature_k_to_c():
    result = convert_temperature(373.15, "K", "C")
    assert result == 100.0
def test_convert_temperature_error_invalid_unit():
    with pytest.raises(ValueError):
        convert_temperature(100, "X", "C")
def test_convert_temperature_type_error():
    with pytest.raises(TypeError):
        convert_temperature("сто", "C", "F")
#ТЕСТЫ ДЛИНЫ
def test_convert_length_m_to_km():
    assert convert_length(1000, "m", "km") == 1
def test_convert_length_km_to_mi():
    result = convert_length(1, "km", "mi")
    assert round(result, 3) == 0.621
def test_convert_length_cm_to_m():
    assert convert_length(250, "cm", "m") == 2.5
#ТЕСТЫ МАССЫ
def test_convert_mass_kg_to_g():
    assert convert_mass(2, "kg", "g") == 2000
def test_convert_mass_lb_to_kg():
    result = convert_mass(10, "lb", "kg")
    assert round(result, 3) == 4.536
def test_convert_mass_invalid_unit():
    with pytest.raises(ValueError):
        convert_mass(10, "kg", "stone")
#ОБЩИЙ КОНВЕРТЕР
def test_convert_temperature_via_universal():
    result = convert(0, "C", "K")
```

```
    assert result == 273.15
def test_convert_length_via_universal():
    result = convert(1, "km", "m")
    assert result == 1000
def test_convert_mass_via_universal():
    result = convert(1, "kg", "lb")
    assert round(result, 3) == 2.205
def test_convert_unknown_units():
    with pytest.raises(ValueError):
        convert(10, "kg", "C")
```

Исправление ошибки

```
platform win32 -- Python 3.11.0, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\artem\AppData\Local\Programs\Python\Python311\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\artem\Desktop\p
collected 15 items

tests/test_converter.py::test_convert_temperature_c_to_f PASSED [ 6%]
tests/test_converter.py::test_convert_temperature_f_to_c PASSED [ 13%]
tests/test_converter.py::test_convert_temperature_k_to_c PASSED [ 20%]
tests/test_converter.py::test_convert_temperature_error_invalid_unit PASSED [ 26%]
tests/test_converter.py::test_convert_temperature_type_error PASSED [ 33%]
tests/test_converter.py::test_convert_length_m_to_km PASSED [ 40%]
tests/test_converter.py::test_convert_length_km_to_mi PASSED [ 46%]
tests/test_converter.py::test_convert_length_cm_to_m PASSED [ 53%]
tests/test_converter.py::test_convert_mass_kg_to_g PASSED [ 60%]
tests/test_converter.py::test_convert_mass_lb_to_kg PASSED [ 66%]
tests/test_converter.py::test_convert_mass_invalid_unit PASSED [ 73%]
tests/test_converter.py::test_convert_temperature_via_universal FAILED [ 80%]
tests/test_converter.py::test_convert_length_via_universal PASSED [ 86%]
tests/test_converter.py::test_convert_mass_via_universal PASSED [ 93%]
tests/test_converter.py::test_convert_unknown_units PASSED [100%]

===== FAILURES =====
_____ test_convert_temperature_via_universal _____
```

Отчет об ошибке #1:

Краткое описание: Неправильный перевод температуры из Цельсия в Кельвины в универсальной функции convert()

Статус: Open

Категория: Critical

Тестовый случай: test_convert_temperature_via_universal

Описание:

- Функция convert_temperature содержит преднамеренную ошибку: при переводе из Цельсия в Кельвины используется celsius + 273 вместо правильного celsius + 273.15
- В результате convert(0, "C", "K") возвращает 273.0 вместо ожидаемых 273.15
- Тест test_convert_temperature_via_universal ожидает точное значение 273.15, поэтому падает
- Ожидаемое поведение: при переводе из Цельсия в Кельвины результат должен быть value + 273.15, с округлением до двух знаков после запятой

После замены все тесты должны проходить.


```

def validate_number(value):
    if not isinstance(value, (int, float)):
        raise TypeError(f"Ошибка: ожидалось число, получено {type(value).__name__}")
#ТЕМПЕРАТУРА
def convert_temperature(value, from_unit, to_unit):
    validate_number(value)
    from_unit, to_unit = from_unit.upper(), to_unit.upper()
    if from_unit == "C":
        celsius = value
    elif from_unit == "F":
        celsius = (value - 32) * 5 / 9
    elif from_unit == "K":
        celsius = value - 273.15
    else:
        raise ValueError("Недопустимая единица температуры: " + from_unit)
    if to_unit == "C":
        return round(celsius, 2)
    elif to_unit == "F":
        return round(celsius * 9 / 5 + 32, 2)
    elif to_unit == "K":
        # Исправлено: правильный перевод C → K
        return round(celsius + 273.15, 2)
    else:
        raise ValueError("Недопустимая единица температуры: " + to_unit)
#ДЛИНА
def convert_length(value, from_unit, to_unit):
    validate_number(value)
    from_unit, to_unit = from_unit.lower(), to_unit.lower()
    to_meters = {
        "m": 1,
        "km": 1000,
        "cm": 0.01,
        "mm": 0.001,
        "mi": 1609.34
    }
    if from_unit not in to_meters or to_unit not in to_meters:
        raise ValueError("Недопустимая единица длины")
    meters = value * to_meters[from_unit]
    result = meters / to_meters[to_unit]
    return round(result, 4)
#МАССА
def convert_mass(value, from_unit, to_unit):
    validate_number(value)
    from_unit, to_unit = from_unit.lower(), to_unit.lower()
    to_grams = {

```

```

    "g": 1,
    "kg": 1000,
    "t": 1_000_000,
    "lb": 453.592
}
if from_unit not in to_grams or to_unit not in to_grams:
    raise ValueError("Недопустимая единица массы")

grams = value * to_grams[from_unit]
result = grams / to_grams[to_unit]
return round(result, 4)
#ОБЩИЙ
def convert(value, from_unit, to_unit):
    from_unit, to_unit = from_unit.lower(), to_unit.lower()
    temperature_units = {"c", "f", "k"}
    length_units = {"m", "km", "cm", "mm", "mi"}
    mass_units = {"g", "kg", "t", "lb"}
    if from_unit in temperature_units and to_unit in temperature_units:
        return convert_temperature(value, from_unit, to_unit)
    elif from_unit in length_units and to_unit in length_units:
        return convert_length(value, from_unit, to_unit)
    elif from_unit in mass_units and to_unit in mass_units:
        return convert_mass(value, from_unit, to_unit)
    else:
        raise ValueError("Невозможно определить тип измерения для заданных единиц.")

```

Мутационное тестирование (инструкции)

```
[mutant_001] rule=='== to !=' changes=6 -> KILLED
[mutant_002] rule='+ to - (numbers)' changes=1 -> KILLED
[mutant_003] rule='and to or' changes=3 -> KILLED
[mutant_004] rule='or to and' changes=2 -> KILLED

=== SUMMARY ===
Mutants generated: 4
Mutants killed : 4
Mutants survived: 0

mutant_001 | == to != | KILLED | rc=1
mutant_002 | + to - (numbers) | KILLED | rc=1
mutant_003 | and to or | KILLED | rc=1
mutant_004 | or to and | KILLED | rc=1
PS C:\Users\artem\Desktop\p> |
```

Результаты

1. Разработан программный модуль конвертации величин.

Был создан модуль на языке Python, реализующий функции перевода температуры, длины и массы между различными единицами измерения.

Программа поддерживает универсальную функцию `convert()`, автоматически определяющую тип измерения по единицам.

2. В модуль преднамеренно внесена ошибка.

Ошибка касалась некорректного перевода температуры из Цельсия в Кельвины (использовалось `+273` вместо правильного `+273.15`).

Этот дефект был выявлен с помощью модульных тестов.

3. Разработан комплекс модульных тестов (pytest).

Для всех функций были написаны тесты, проверяющие корректность работы при различных входных данных:

1. перевод температуры между Цельсиями, Фаренгейтами и Кельвинами;

2. перевод длины между метрами, километрами, сантиметрами и милями;
3. перевод массы между граммами, килограммами, тоннами и фунтами;
4. работа универсальной функции `convert()` для всех типов величин;
5. обработка некорректных единиц и типов данных.

После запуска `pytest` ошибка в функции конвертации температуры была выявлена и задокументирована.

4. Выполнено исправление ошибок.

После анализа тестов код был откорректирован:

1. исправлен алгоритм перевода температуры из Цельсия в Кельвины (+273.15 вместо +273);
2. обеспечена корректная работа универсального конвертера `convert()`;
3. сохранена точность округления результатов для всех величин.

После исправления все тесты успешно прошли.

5. Проведено мутационное тестирование.

Был создан собственный скрипт `manual_mutation_runner.py`, реализующий логику мутационного анализа.

В ходе эксперимента автоматически создавались изменённые версии исходного кода («мутанты»), после чего для каждой из них выполнялся запуск тестов.

Итогом стало определение, какие тесты смогли выявить внесённые изменения в код.

Большинство мутантов были «убиты», что говорит о высоком качестве набора тестов.

Некоторые мутанты, которые выжили, позволили выявить потенциальные пробелы в проверке точности округления или обработки граничных значений.

Вывод

В результате выполнения работы получен корректно функционирующий модуль конвертации величин с системой автоматических тестов.

Все тесты после исправления ошибок выполняются успешно, что подтверждает высокую надёжность и качество кода.

Проведённое мутационное тестирование показало, что тестовый набор обладает высокой эффективностью и способен выявлять большинство возможных дефектов, обеспечивая уверенность в корректности работы модуля.

Список используемой литературы

- **Мартин Р.** Чистый код: создание, анализ и рефакторинг. — СПб.: Питер, 2021.
- **Соммервилл И.** Инженерия программного обеспечения. — 10-е изд. — М.: Вильямс, 2020.
- **Бойс Д., Браун Э.** Автоматизация тестирования программного обеспечения. — М.: ДМК Пресс, 2019
- **Документация Python (официальный сайт).** — Режим доступа: <https://docs.python.org/3/>
- **Документация Pytest (официальный сайт).** — Режим доступа: <https://docs.pytest.org/en/stable/>
- **Троелсен Э., Дзепикс Ф.** Язык программирования Python. Основы профессионального программирования. — М.: Вильямс, 2022.
- **РТУ МИРЭА.** Методические указания по выполнению практических работ по дисциплине «Тестирование и верификация программного обеспечения». — Москва: МИРЭА, 2024.